

AFPL CXM Generator - Technical Documentation

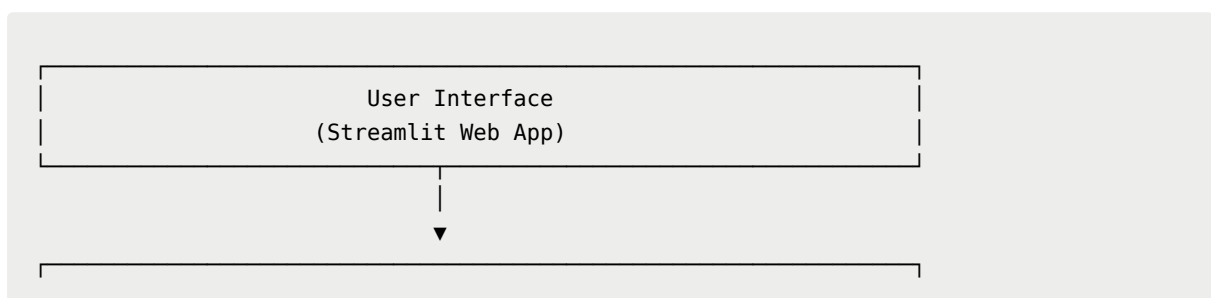
Project Overview

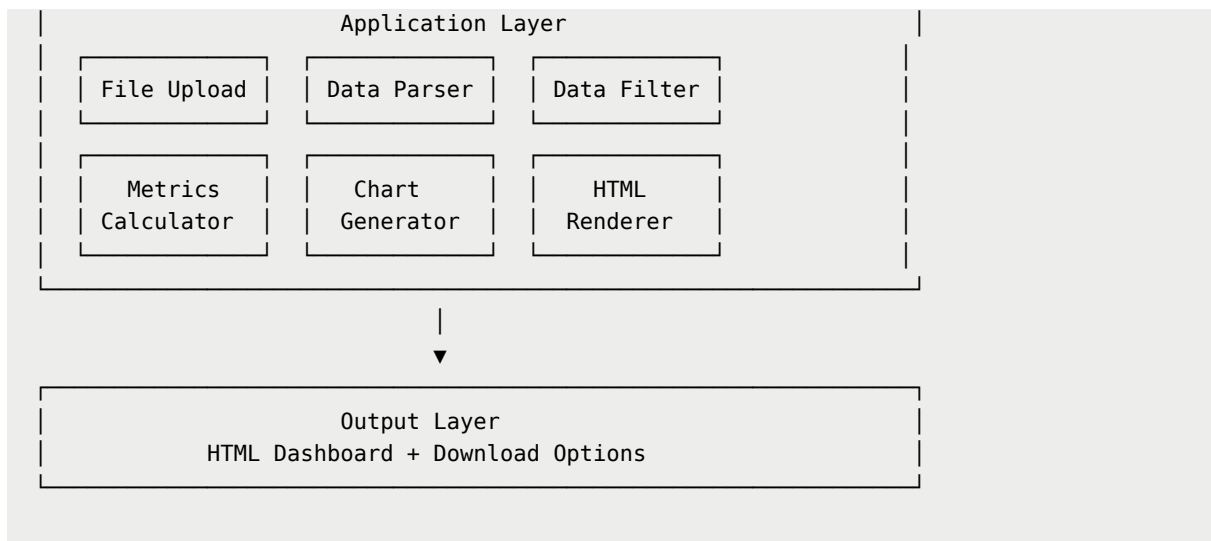
The **AFPL CXM Generator** is a Streamlit-based web application that generates dynamic Customer Experience Management (CXM) dashboards for Anandmayee Forgings Pvt Ltd. The application reads Excel workbooks containing PO data, dispatch records, items data, and survey feedback, then produces interactive HTML dashboards with KPIs, charts, and operational insights for specific customers.

Table of Contents

1. [System Architecture](#)
 2. [Installation & Setup](#)
 3. [File Structure](#)
 4. [Configuration Files](#)
 5. [Core Application Logic](#)
 6. [Data Processing Pipeline](#)
 7. [Template System](#)
 8. [Deployment Guide](#)
 9. [Troubleshooting](#)
 10. [Maintenance & Extensions](#)
-

System Architecture





Technology Stack

Component	Technology
Frontend	Streamlit, HTML/CSS, Chart.js
Backend	Python 3.8+
Data Processing	Pandas, OpenPyXL
Charting	Chart.js (embedded)
Styling	Custom CSS, Google Fonts

Installation & Setup

Prerequisites

- Python 3.8 or higher
- pip package manager
- Git (optional)

Step 1: Clone/Download the Project

```
cd D:\\Automation_Projects\\afpl_cxm_app
```

Step 2: Create Virtual Environment

```
python -m venv venv
```

Activate it:

Windows:

```
venv\\Scripts\\activate
```

Linux/Mac:

```
source venv/bin/activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Run the Application

```
streamlit run app.py
```

The app will open at <http://localhost:8501>

File Structure

```
afpl_cxm_app/
├── app.py                # Main Streamlit application
├── requirements.txt      # Python dependencies
├── packages.txt          # System packages (for deployment)
├── template.html         # HTML dashboard template
├── template - org.html   # Original template (backup)
├── chart.umd.min.js      # Chart.js library (embedded)
├── .streamlit/
│   └── config.toml       # Streamlit configuration
└── assets/
    └── favicon.png       # Company logo
```

File Descriptions

File	Purpose
<code>app.py</code>	Main application logic, UI components, data processing

File	Purpose
<code>requirements.txt</code>	Python packages: streamlit, pandas, openpyxl
<code>packages.txt</code>	System-level packages for Linux deployment
<code>template.html</code>	HTML/CSS/JS template for dashboard output
<code>chart.umd.min.js</code>	Chart.js library (version 4.4.0)
<code>.streamlit/config.toml</code>	Streamlit theme and server settings
<code>assets/favicon.png</code>	Company logo displayed in header

Configuration Files

`.streamlit/config.toml`

```
[theme]
base = "light"
primaryColor = "#FF6B1A"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F8F9FC"
textColor = "#1E293B"
font = "sans serif"

[server]
maxUploadSize = 100

[browser]
gatherUsageStats = false
```

Settings Explained:

- `maxUploadSize = 100` - Maximum file upload size in MB
- `primaryColor = "#FF6B1A"` - Brand orange color (#ef4444 is used in CSS)
- `gatherUsageStats = false` - Disables Streamlit telemetry

Core Application Logic

Main Application Flow

1. User uploads Excel **file**
↓
2. System classifies sheets (active_po, dispatch, items, survey)
↓
3. User selects customer **from** extracted **list**
↓
4. User selects reporting period (All/Yearly/Quarterly/Monthly/Weekly/Custom)
↓
5. User clicks "**Generate Dashboard**"
↓
6. System builds data structures (active POs, dispatch history, items, NPS)
↓
7. System applies date filters
↓
8. System renders HTML dashboard
↓
9. User can download HTML **or print** to PDF

Key Functions Reference

Function	Location	Purpose
<code>load_workbook_sheets()</code>	<code>app.py</code>	Reads Excel, classifies sheets
<code>classify_sheet()</code>	<code>app.py</code>	Identifies sheet type by column headers
<code>extract_customers()</code>	<code>app.py</code>	Gets unique customer names
<code>build_active_pos()</code>	<code>app.py</code>	Creates active PO records
<code>build_dispatch_history()</code>	<code>app.py</code>	Creates dispatch history
<code>build_items()</code>	<code>app.py</code>	Creates line item records
<code>extract_nps_csat_data()</code>	<code>app.py</code>	Parses survey data
<code>filter_*_by_date()</code>	<code>app.py</code>	Date-based filtering
<code>render_html()</code>	<code>app.py</code>	Injects data into template

Sheet Classification Rules

The system identifies sheets by checking column headers:

Sheet Type	Must Have	Should Have
active_po	"PO DATE"	PO NO., Value, PO Approval Status, Person Name
dispatch	"PO Number", "PO Date"	LAST INVOICE STATUS, Payment due on, Dispatch Status
items	"PO Item Name"	MATERIAL, ORDER QTY., BAL. QTY., VALUE
survey	"Date", "customer"	Promoters, Passives, Detractors, Percentage

Excel Date Handling

The system handles multiple date formats:

- Standard dates (YYYY-MM-DD)
- Excel serial numbers (e.g., 45974 → 2025-11-15)
- Various formats: `%d.%m.%Y`, `%d/%m/%Y`, `%d-%m-%Y`, `%m/%d/%Y`

KPI Calculations

KPI	Calculation	Data Source
Open Orders	Active POs not delivered	active_po sheet
Delivered Orders	POs with delivered = "Yes"	dispatch sheet
Delayed Orders	POs with <code>is_delayed = True</code>	active_po sheet
NPS Score	(Promoters - Detractors) / Total × 100	survey sheet
CSAT Score	Average of percentage scores	survey sheet

Template System

`template.html` Structure

The template uses `{{PLACEHOLDER}}` syntax for dynamic content injection:

Placeholder	Source	Description
<code>{{COMPANY}}</code>	Hardcoded	"Anandmayee Forgings"
<code>{{COMPANY_LOGO}}</code>	assets/favicon.png	Base64 encoded logo

Placeholder	Source	Description
{{CUSTOMER}}	User selection	Customer name
{{KAM}}	Extracted from data	Key Account Manager
{{PERIOD}}	User selection	Reporting period label
{{HERO_SUB}}	Generated	Dynamic hero text
{{ITEM_COUNT}}	Calculated	Number of line items
{{GEN_DATE}}	System date	Generation date
{{DATA_JSON}}	Data structures	All chart/table data as JSON

Dashboard Sections

1. **Executive Summary** - 5 key KPIs
2. **Order Tracking** - Aging analysis, alerts, status distribution
3. **Item-Level Execution** - Line item tracking, dispatch volume
4. **Complaint Management** - Hidden (placeholder data only)
5. **Voice of Customer** - NPS/CSAT trends and distribution
6. **Collections** - Hidden (accounts data pending)

Deployment Guide

Local Deployment (Windows)

```
# Install Python 3.8+
# Clone/copy project to D:\\Automation_Projects\\afpl_cxm_app
cd D:\\Automation_Projects\\afpl_cxm_app
python -m venv venv
venv\\Scripts\\activate
pip install -r requirements.txt
streamlit run app.py
```

Cloud Deployment (Streamlit Community Cloud)

1. Push code to GitHub repository
2. Go to share.streamlit.io

3. Connect GitHub account
4. Select repository and branch
5. Set `app.py` as main file
6. Deploy

Linux Server Deployment (with packages.txt)

`packages.txt` contains required system libraries for headless browsers (if using PDF generation):

```
sudo apt-get update
xargs -a packages.txt sudo apt-get install -y
```

Docker Deployment (Optional)

Create a `Dockerfile`:

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

Build and run:

```
docker build -t afpl-cxm .
docker run -p 8501:8501 afpl-cxm
```

Expected Excel Format

Required Columns by Sheet

Active PO Sheet:

Customer, PO NO., PO DATE, Value, PO Approval Status, Person Name, Status/Wo No, Delivery Time (Req. By Cust.) CDD, Delivery Time (Given By PPC) EDD, Month

Dispatch Sheet:

Customer Name, PO Number, PO Date, LAST INVOICE STATUS, Payment due on, Dispatch Status, Delivered to Customer, Month

Items Sheet:

CUSTOMER NAME, PO, PO Item Name, ORDER QTY., BAL. QTY., VALUE, DISPATCH DATE, KAM Name, MATERIAL, Work Order (W.O.), CDD, EDD

Survey Sheet (Optional):

Date, customer, Promoters, Passives, Detractors, Percentage

Troubleshooting

Common Issues and Solutions

Issue	Likely Cause	Solution
"Sheet could not be classified"	Wrong column headers	Ensure columns match expected names
Dates appear as numbers	Excel serial dates	Already handled by <code>to_date_str()</code>
No customers found	"Customer" column missing	Check sheet has Customer/Customer Name column
NPS shows no data	No survey sheet or missing columns	Add survey data or ignore
Chart.js errors	Missing chart.umd.min.js	Ensure file exists in root directory
Logo not showing	Missing assets/favicon.png	Place logo in assets folder

Debug Mode

Add this at the top of `app.py` to enable debug logging:

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

Testing with Sample Data

Create a minimal Excel file with:

- One sheet named "active_po" with columns: Customer, PO NO., PO DATE, Value
- At least one customer record
- Upload and test

Maintenance & Extensions

Adding New KPIs

1. Add calculation in the `M` object inside `template.html` JavaScript
2. Add HTML element with appropriate `id`
3. Update the `kpi-grid` section in template

Adding New Chart Types

1. Add `<canvas>` element in template with unique ID
2. Add Chart.js initialization code in template JavaScript
3. Ensure data is included in `DATA_JSON`

Extending Sheet Classification

Add new signature to `SHEET_SIGNATURES` dictionary:

```
SHEET_SIGNATURES = {
    "new_sheet_type": {
        "must_have": ["Column1", "Column2"],
        "should_have": ["Column3", "Column4"],
    },
}
```

```
# ... existing  
}
```

Adding Date Filter for NPS

NPS filtering is already implemented via `filter_nps_by_date()`. Ensure survey sheet has proper date column.

Contact & Handover Notes

Project Location

```
D:\\Automation_Projects\\afpl_cxm_app
```

Key Files for New Developer

1. `app.py` - Start here to understand main flow
2. `template.html` - Understand dashboard structure
3. `requirements.txt` - For environment setup

Dependencies Summary

```
streamlit    # Web framework  
pandas       # Data manipulation  
openpyxl     # Excel file reading
```

External Dependencies

- Chart.js (v4.4.0) - bundled as `chart.umd.min.js`
- Google Fonts (Inter, Fraunces, JetBrains Mono)

Assumptions

- Excel files follow the column naming conventions
- Customer names are consistent across sheets
- Dates are either proper dates or Excel serial numbers
- Survey data, if present, includes NPS calculation columns

Known Limitations

1. Collections/Accounts data is placeholder (hidden section)
2. Complaint Management uses placeholder data
3. Large Excel files (>100MB) may cause timeout
4. Chart rendering requires JavaScript enabled

Future Enhancement Suggestions

1. Add database backend for persistent storage
 2. Implement user authentication
 3. Add email report delivery
 4. Create API endpoints for automated report generation
 5. Add more export formats (PDF, Excel, PowerPoint)
 6. Implement real-time data refresh
-

Appendix

Environment Variables (Optional)

Create `.streamlit/secrets.toml` for sensitive data:

```
# Not currently used, but预留 for future features
API_KEY = "your-api-key"
DB_CONNECTION = "connection-string"
```

Useful Commands

```
# Run with specific port
streamlit run app.py --server.port=8080

# Run without browser opening
streamlit run app.py --server.headless=true

# Check Streamlit version
streamlit version
```

License

Internal use only - Anandmayee Forgings Pvt Ltd

Documentation Version: 1.0

Last Updated: 2026-06-01

Maintainer: Harpinder Singh